BHARATI VIDYAPEETH'S COLLEGE OF ENGINEERING, LAVALE, PUNE

DEPARTMENT OF COMPUTER ENGINEERING

DATA STRUCTURE MINI PROJECT

IMPLEMENTATION OF QUEUE USING TWO STACKS

[Stack | Queue | Python]

Submitted by	RUDRAKSHI KOUL
Roll Number	09
Class / Semester	SE-B COMP | 2026–27 | Semester I
Subject	Data Structures (Python)
Academic Year	2026–2027

Submitted to the Department of Computer Engineering

BHARATI VIDYAPEETH'S COLLEGE OF ENGINEERING, LAVALE, PUNE

DEPARTMENT OF COMPUTER ENGINEERING

CERTIFICATE

This is to certify that the mini project report entitled “Implementation of Queue Using Two Stacks” submitted by Rudrakshi Koul (Roll No. 09) is work carried out by her during the academic year 2026–2027, in partial fulfillment of the requirements for the course Data Structures, at Bharati Vidyapeeth's College of Engineering, Lavale, Pune.

Project Guide **Head, Department of Computer Engineering**

Date: _____

Place: Lavale, Pune

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide for valuable guidance, encouragement, and support during the completion of this mini project. The suggestions and feedback provided helped me understand the data structure concepts and their practical implementation.

I am thankful to the Head of the Department of Computer Engineering and the faculty of Bharati Vidyapeeth's College of Engineering, Lavale, Pune, for providing the academic support and resources needed for this work.

Finally, I thank my family and friends for their encouragement and support throughout the project.

Rudrakshi Koul
Roll No. 09

ABSTRACT

This project implements a First-In, First-Out (FIFO) queue using two stacks. A queue normally inserts items at the rear and removes items from the front. The implementation uses an input stack for enqueue operations and an output stack for dequeue and peek operations. When the output stack is empty, elements are transferred from the input stack to reverse their order, making the oldest inserted element available first. The program provides a menu-driven interface for inserting, removing, viewing, and displaying queue items. It demonstrates how one abstract data type can be implemented using another and explains the time complexity of the main operations.

Keywords: Data Structures, Stack, Queue, Python, algorithm, implementation, testing

TABLE OF CONTENTS

Sr. No.	Content	Page
1	Introduction, Problem Statement and Objectives	6
2	Literature Survey	6
3	Algorithm and UML / Flow Design	6
4	System Requirements	6
5	System Design	7
6	System Code	7
7	Test Cases and Results	8
8	Applications	9
9	Future Scope	10
10	Conclusion	10
11	References	10

1. Introduction, Problem Statement and Objectives

A queue is a linear data structure that follows the First-In, First-Out (FIFO) principle. The element inserted first is the first one removed. Standard queue operations are enqueue, dequeue, peek, and checking whether the queue is empty.

A stack follows Last-In, First-Out (LIFO). This project demonstrates that a queue can be built using two stacks by carefully transferring elements between them.

Problem statement: Implement queue operations using only stack behaviour while preserving FIFO order.

Objectives:

- Implement enqueue and dequeue using two stacks.
- Support peek and display operations.
- Handle underflow (dequeue or peek on an empty queue).
- Analyse the time complexity of each operation.

2. Literature Survey

Classic data-structure texts describe stacks and queues as fundamental linear abstract data types. A stack permits insertion and deletion at one end, while a queue inserts at the rear and deletes at the front.

A common two-stack queue implementation stores new elements in an input stack and reverses them into an output stack only when the output stack becomes empty. This amortized approach avoids transferring elements on every dequeue.

3. Algorithm and UML / Flow Design

Enqueue: push the new item onto the input stack.

Dequeue: if the output stack is empty, pop all items from the input stack and push them onto the output stack. If both stacks are empty, report an empty queue; otherwise pop from the output stack.

Peek: perform the same transfer if required, then read the top of the output stack without removing it.

Display: show the output stack in reverse order followed by the input stack, giving the queue from front to rear.

Flow of the program:

Start → Show menu → Read choice → Perform Enqueue / Dequeue / Peek / Display → Repeat until Exit → Stop

UML concept: The user interacts with QueueUsingTwoStacks. The class owns in_stack and out_stack and provides enqueue(), dequeue(), peek(), is_empty(), and display().

4. System Requirements

Hardware: computer or laptop with a modern processor, at least 2 GB RAM, and sufficient storage.

Software: Windows, Linux, or macOS; Python 3.x; any code editor or terminal. No third-party libraries are required.

5. System Design

The system is implemented as a class named QueueUsingTwoStacks. The input stack receives newly enqueued values. The output stack supplies the front value. A transfer is performed only when the output stack has no elements.

Existing approach: a queue may be implemented using an array or linked list with direct front/rear management. **Proposed approach:** use two stack containers and transfer elements lazily, demonstrating the relationship between abstract data types.

Complexity: enqueue is O(1); dequeue and peek are O(1) when the output stack is non-empty and O(n) when a transfer is needed. Each item is transferred at most once, so enqueue and dequeue have amortized O(1) time. Space complexity is O(n).

Working example (stacks written from bottom to top):

Operation	in_stack (bottom to top)	out_stack (bottom to top)	Result
enqueue(A)	[A]	[]	-
enqueue(B)	[A, B]	[]	-
enqueue(C)	[A, B, C]	[]	-
dequeue()	[]	[C, B]	A
enqueue(D)	[D]	[C, B]	-
dequeue()	[D]	[C]	B
dequeue()	[D]	[]	C
dequeue()	[]	[]	D

6. System Code

```
class QueueUsingTwoStacks:
    """Queue implemented using two stacks."""

    def __init__(self):
        self.in_stack = []
        self.out_stack = []

    def enqueue(self, item):
        """Insert an item at the rear of the queue."""
        self.in_stack.append(item)
        print(f"{item} inserted into the queue.")

    def _transfer(self):
        """Move items only when the output stack is empty."""
        while self.in_stack:
            self.out_stack.append(self.in_stack.pop())

    def dequeue(self):
        """Remove and return the front item."""
        if not self.out_stack:
            self._transfer()
        if not self.out_stack:
            print("Queue is empty.")
            return None
        item = self.out_stack.pop()
        print(f"Removed item: {item}")
        return item
```

```
    def peek(self):
        """Return the front item without removing it."""
        if not self.out_stack:
            self._transfer()
        if not self.out_stack:
            print("Queue is empty.")
            return None
        return self.out_stack[-1]

    def is_empty(self):
        return not self.in_stack and not self.out_stack

    def display(self):
        """Display queue contents from front to rear."""
        front_to_rear = list(reversed(self.out_stack)) + self.in_stack
        if not front_to_rear:
            print("Queue is empty.")
        else:
            print("Front ->", " | ".join(map(str, front_to_rear)), "<- Rear")


def main():
    queue = QueueUsingTwoStacks()
    while True:
        print("\n--- Queue Using Two Stacks ---")
        print("1. Enqueue")
        print("2. Dequeue")
        print("3. Peek")
        print("4. Display")
        print("5. Exit")
        choice = input("Enter choice: ").strip()

        if choice == "1":
            value = input("Enter value to enqueue: ")
            queue.enqueue(value)
        elif choice == "2":
            queue.dequeue()
        elif choice == "3":
            value = queue.peek()
            if value is not None:
                print("Front item:", value)
        elif choice == "4":
            queue.display()
        elif choice == "5":
            print("Exiting program.")
            break
        else:
            print("Invalid choice. Please try again.")


if __name__ == "__main__":
    main()
```

7. Test Cases and Results

Test	Input / Action	Expected Result	Status
1	Enqueue A, B, C; display	Front -> A | B | C <- Rear	Pass
2	Dequeue after inserting A, B, C	Removed item: A	Pass
3	Peek after inserting 10 and 20	Front item is 10; queue unchanged	Pass
4	Dequeue from an empty queue	Queue is empty.	Pass
5	Enqueue 1, 2; dequeue; enqueue 3; dequeue 3 times	Removed 1, 2, 3, then Queue is empty.	Pass

Expected result: the FIFO order is maintained for mixed enqueue and dequeue operations.

Sample Output

```
--- Queue Using Two Stacks ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter choice: 1
Enter value to enqueue: A
A inserted into the queue.

--- Queue Using Two Stacks ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter choice: 1
Enter value to enqueue: B
B inserted into the queue.

--- Queue Using Two Stacks ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter choice: 4
Front -> A | B <- Rear

--- Queue Using Two Stacks ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter choice: 2
Removed item: A

--- Queue Using Two Stacks ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter choice: 3
Front item: B

--- Queue Using Two Stacks ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter choice: 5
Exiting program.
```

8. Applications

The queue concept is useful in CPU scheduling, printer queues, customer-service systems, breadth-first search, network packet handling, and task processing. The two-stack implementation is also useful for

learning how data structures can be composed.

9. Future Scope

The program can be extended with automated unit tests, a graphical interface, input validation for numeric values, and a fixed-capacity option. It can also be compared with queue implementations based on arrays, linked lists, and circular buffers.

10. Conclusion

The mini project successfully demonstrates a FIFO queue implemented with two LIFO stacks. The transfer operation reverses element order when needed, allowing the oldest element to be removed first. The program covers the core queue operations and illustrates amortized time complexity.

11. References

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. Introduction to Algorithms. MIT Press.

2. Goodrich, M. T., Tamassia, R., and Goldwasser, M. H. Data Structures and Algorithms in Python. Wiley.

3. Python Software Foundation. Python 3 Documentation, https://docs.python.org/3/